



8

21

2520 sp

نظري

كلية الهندسة المعلوماتية

السنة الخامسة - هندسة البرمجيات

Components

د. باسم قصيبة



RB Informatics; 16/12/2024

هندسة برمجيات ٣

تحدثنا في المحاضرة السابقة عن MDE وكان الهدف منها إنشاء software process.

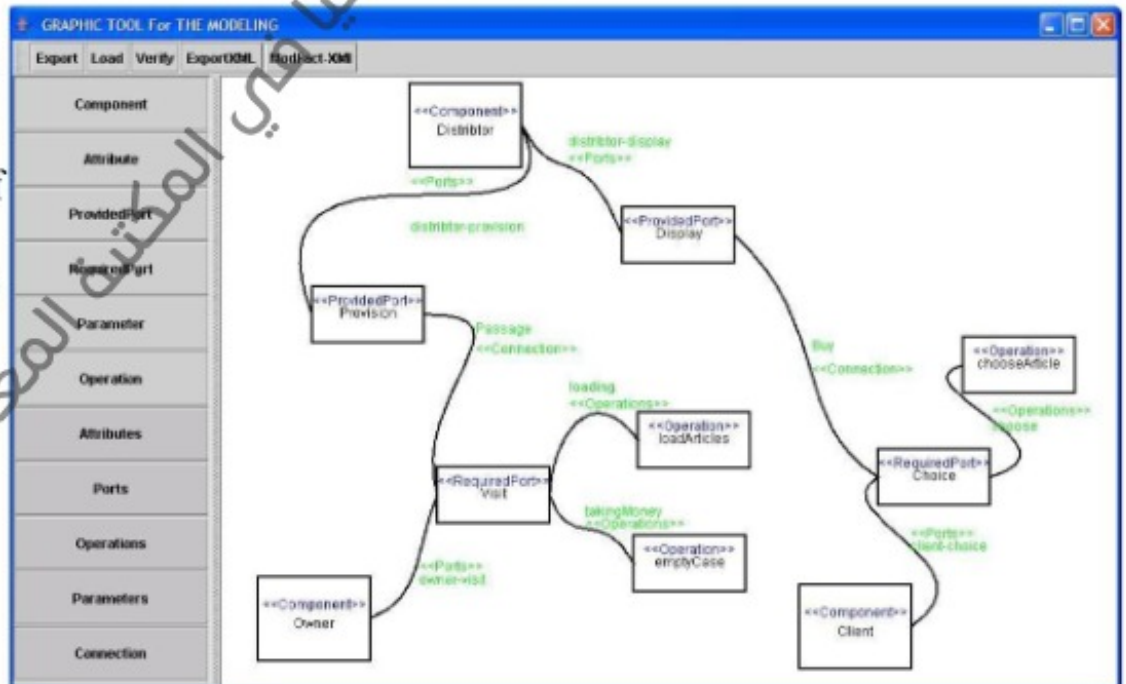
Software Process

تتكون من عدة Steps وكل step له Meta Model خاص به بحيث يقدم ال Meta Model نمودجاً يصف هذه ال Step وما الأدوات الواجب استخدامها في هذه ال Step، كما يوجد علاقات تربط بين ال Meta Models. يوجد أدوات لتوصيف ال Meta Models مثل برنامج Graphic tool for the modeling.

أمثلة عن برنامج Graphic tool for the modeling (الاطلاع):

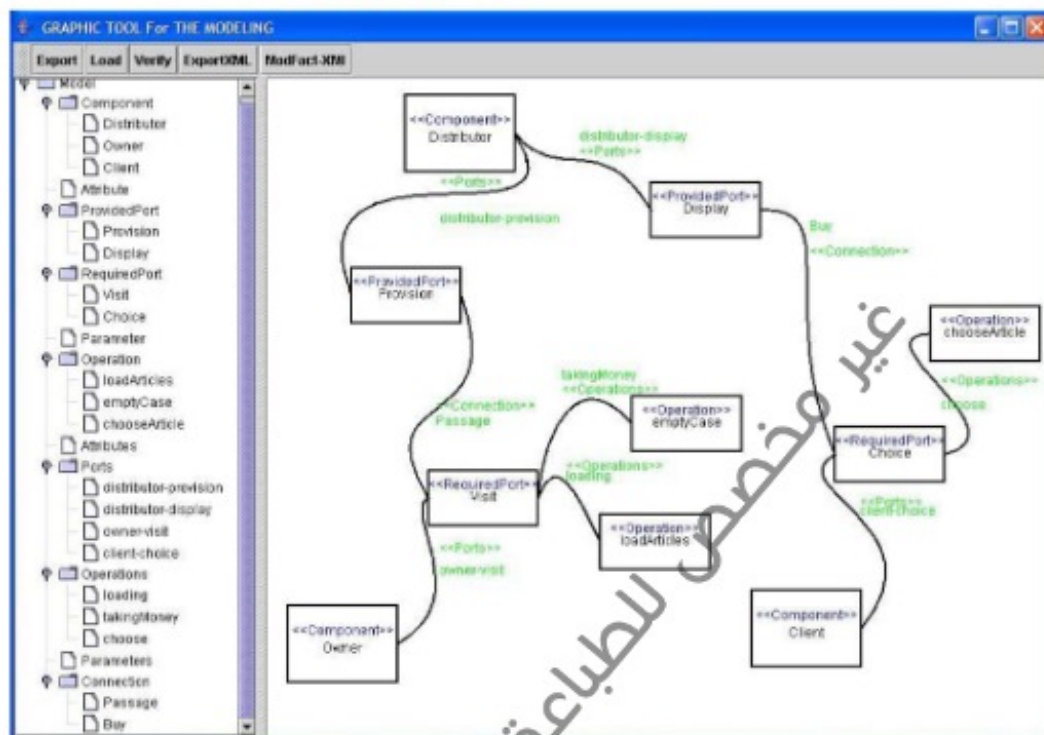
Graphic tool <1>

- Domain concepts → Buttons set
- Handling of model elements → Drawing board



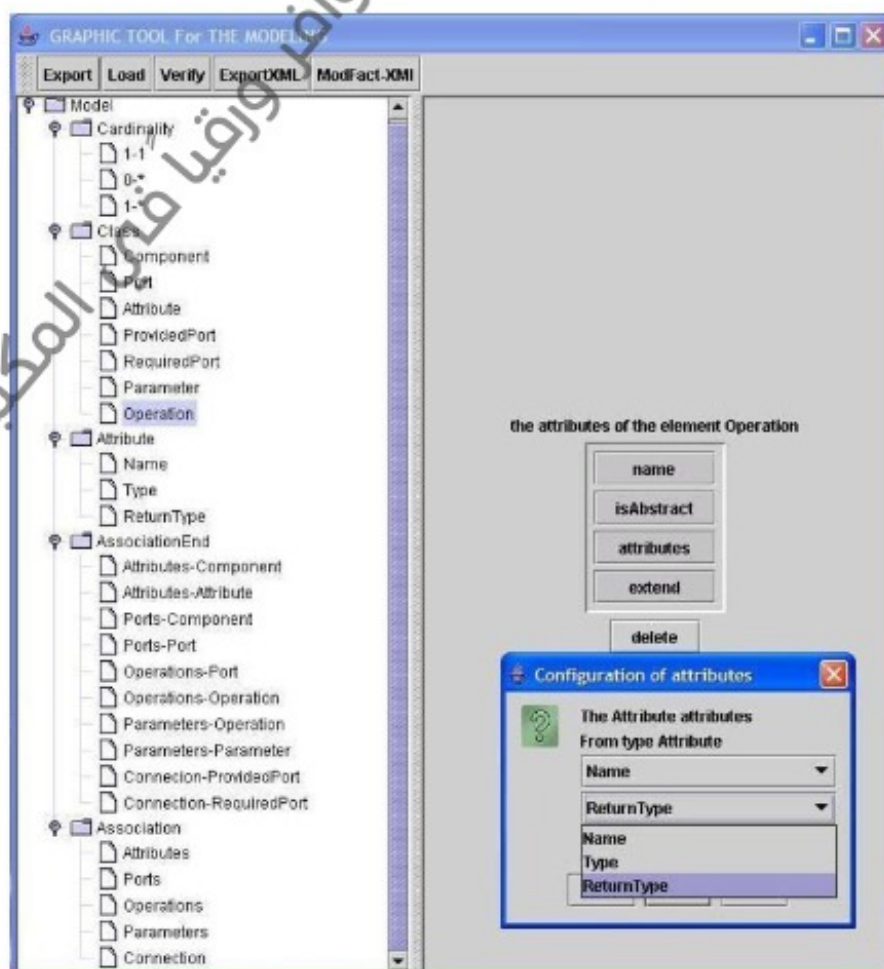
Graphic tool <<2>>

- Domain concepts → Tree
- Handling of model elements → Drawing Board



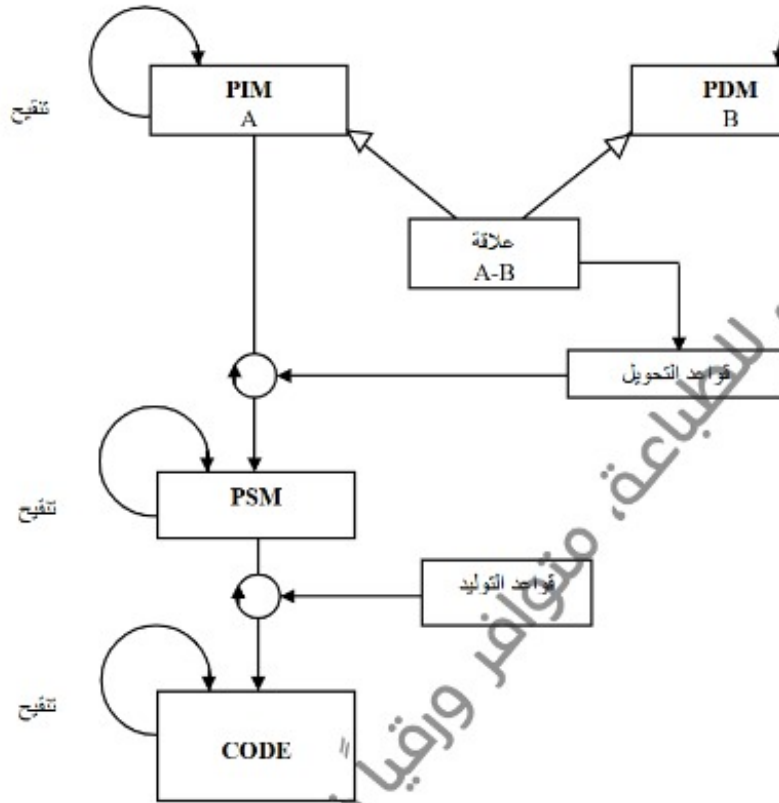
Graphic tool for modeling of application domains

- lightMOF concepts → Tree
- Handling of model elements → Form set



MDA (Model Driven Architecture)

- ال OMG يقترح ال MDA بهدف الحفاظ على الاستثمارات الفكرية والمالية المنجزة أثناء تطوير النظم وذلك عندما تتغير تقنية التنفيذ أو ترتقي.
- الفكرة الرئيسية لل MDA هي الفصل أثناء بناء النظام بين التصميم المهني والتحقيق التقني لهذا التصميم.



MDA هي حالة خاصة من MDE حيث أن MDA تتوقف عند مشكلة تعدد التقنيات (مثلاً: تم إنشاء نظام وتم توليده باستخدام java, فيمكن غداً تطويره باستخدام ++C...) OMG هي اختصار لـ Object Management Group وهي منظمة تهدف إلى وضع معايير تقنية لتطوير البرمجيات وللاسيما النماذج و المخططات (ك MDA و UML وغيرها...)

مفردات ال MDA

- PIM (Platform Independent Model): يمثل النموذج المهني المستقل عن أي تقنية تنفيذ ويعبر عن مشكلة باستخدام مفاهيم مجالها (النموذج مستقل عن منصة التطوير).
 - PDM (Platform Definition Model): يمثل تعريف لمفاهيم تقنية التنفيذ المستخدمة لتحقيق النموذج المهني (الموديل المرتبط بالتقنية).
 - PSM (Platform Specific Model): يحوي نفس كائنات ال PIM مع تفاصيل خاصة بتقنية التنفيذ المختارة (موديل خاص بهذه التقنية (يجب أن نجد طريقة للدمج لينتج لدينا PSM)).
- يجب أن نجد meta model يوصف التقنيات كلها و meta model يوصف آخر نموذج حصلنا عليه من ال MDE.

ملحوظة: MDA بدأ بعد مرحلة ال MDE.

مثلاً أقوم بعمل code generator لآخر نموذج ضمن MDE، نريد أن نفك ال code generate وتوفير إمكانية لأتمتته، ولنستطيع أتمتته: في حال كان لدي تقنية C++ يجب أن يكون لها meta model، أو تقنية java لها meta model...

نريد ربط المفهوم process بتقنية (task, process, class, component) وذلك وفقاً للتقنيات: (غرضي التوجه ← class، تقنية ADA هي task). وهذا هو تجريد التقنيات لهذا المستوى وعلاقتها مع هذا المشروع، وهذا كله في طور التطبيق.

Components

جاءت بثلاث ميزات:

1. توجه نحو معمارية التطبيقات (يمكن تغيير سلوك التطبيق من خلال تغيير البيئة).
2. تسمح بالتصريح عن الخدمات الغير وظيفية بدلاً من برمجتها.
3. تقدم آلية لنشر وتوزيع التطبيقات المعتمدة على المكونات البرمجية.

شرح الميزة الأولى:

المشكلة الأساسية للنموذج غرضي التوجه هو أنه عند التعديل على البرنامج يجب فتح الكود وتغيير العلاقات بين الأغراض وإعادة التجميع والتنفيذ.

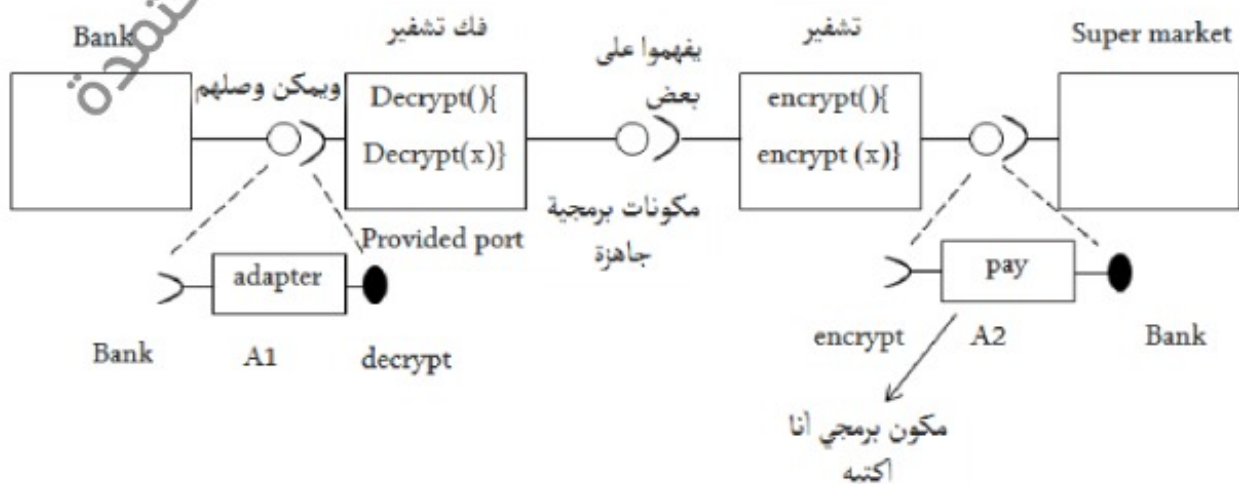
- السبب: لأنه لا يوجد علاقة (عمليات) صريحة لتبادل ال reference بين الأغراض.
- الحل: المكونات البرمجية (Components) أضافت فعلياً هذه العمليات (لتبادل ال references)

معمارية التطبيقات: تفسر على عدة أعماق، العمق الظاهري والعمق الباطني.

العمق الظاهري:

مثال: بنك يقدم خدمات.

يمكن تمثيل المكون البرمجي على أنه server، كل مجموعة operation من نوع معين تسمى facet أو provided port (العمليات المقدمة).



- Supermarket يستخدم خدمات البنك مثل `pay(int x)` عند شراء أي بضاعة، يستخدم هذه الخدمة ليأخذ نقود من حساب الزبون (على فرض `x` تحتوي الكثير من البيانات مثل المبلغ ، رقم الحساب...)، وليتم إضافتها لحساب ال `supermarket`.

على فرض لدينا مشكلة وهي الأمن، لا يجب إرسال `x` بشكل مكشوف.

- وبالتالي نحن نحتاج إلى إضافة مكون للتشفير وطالما أنني أعمل بتقنية المكونات البرمجية فأبحث على النت أو ضمن مكتبة الشركة (خبرة الشركة) على مكون للتشفير ومكون لفك التشفير وأوصلهم مع بعض، وبهذه الطريقة لم أقم بتغيير نظام المؤسسة وإنما قمت بإضافة عليه.
- هذه هي سياسة الاستخدام، المكونات البرمجية تتجه نحو معمارية التطبيقات أي يمكن تغيير سلوك التطبيق من خلال تغيير بنيته.
- تم تغيير سلوك النظام من سلوك غير آمن إلى سلوك آمن من خلال إضافة مكونات برمجية، أي تغيير في البنية.
- هذه العملية ليست بهذه البساطة لأن كاتب مكون التشفير أو فك التشفير لا يعلم كيف ستستخدم.
- لا يمكن ربط المكونات البرمجية دون أن تكون ال `interface` متناسقة، ولذلك هنا نستخدم ال `Adapter` الذي سميناه `A1` ونكتبه ليتوافق مع المكونات ويتم ربطهم، ويجب أن يحتوي على `function` وحتمًا اسمه `decrypt` لأن ال `provider` الخاص به يقدم `decrypt` وينفيس الطريقة نستخدم `A2`، ولندخل في تفاصيل ال `Adapter`.

المكون البرمجي يستطيع أن يتحسس محيطه، ماذا يعني ذلك؟

```
Class A1{
    Decrypt(x){
        Bank b = CCM.getBank();
        b.pay(x); }
}
```

داخل فك الشفرة
نقوم بعمل `pay`



- يعرف ماهي ال `ports` التي يملكها.

```
Class A2{
    pay(x){
        Encryptor e = CCM.getEncryptor();
        e.encrypt(x);}
}
```

على فرض لدينا بيئة `CCM`
`CORBA Component Model`
Supermarket ينادي `A2` ← `pay`، فيتم استدعاء `required port`
هو الذي يلتقط ال `reference: CCM.getEncryptor()`

- نحن نعلم أنه لدينا هذا ال port وهو من نوع Encryptor ، وهنا تمت إعادة غرض وقمنا من خلاله بعملية $\text{eccrypt}(x)$.
- وصلنا لعملية فك التشفير، يريد استخدام خدمة البنك وهي ال pay ← سيمر A1 ← decrypt نحضر reference من البنك ومن ثم بعد عملية فك التشفير عادت صحيحة ووصلت للطرف الثاني صحيحة وكما يجب، وبتسلسل غرضي التوجه.

■ مهمة ال Adapter هنا تصنع المكونات البرمجية مع بعضها.

خلاصة: لا يجوز ربط 2port مع بعض دون interface وعملياته الأساسية تبادل المراجع. Required port يستقبل مرجع من نوع معين.

توضيح للمكونات البرمجية:

- سابقاً عند شراء جهاز حاسب نذهب للمحل ونطلب المكونات التي نريدها ويقوم صاحب المحل بعملية تجميع القطع، وبنفس الأسلوب ضمن البرمجة الخطوة التالية بعد ال OOP كانت السعي أن تكون البرمجة مؤتمتة.
- عملية بناء التطبيق هي assembly لمجموعة من العمليات مثل (, search , sort) فبدلاً من كتابة كل العمليات يدوياً أقوم بعملية استعارة ويتم بناء البرنامج، فتكون هذه العمليات (المكونات) على الرف on staff موجودين، أخذهم ويتم تركيبهم وتنفيذهم فوراً.
- وليسوا بحاجة إلى إعادة ترجمة للتطبيق بالكامل لأن لهم عمليات صريحة لتبادل المراجع، و الترجمة كانت فقط لوضع هذا ال linking بين ال objects.

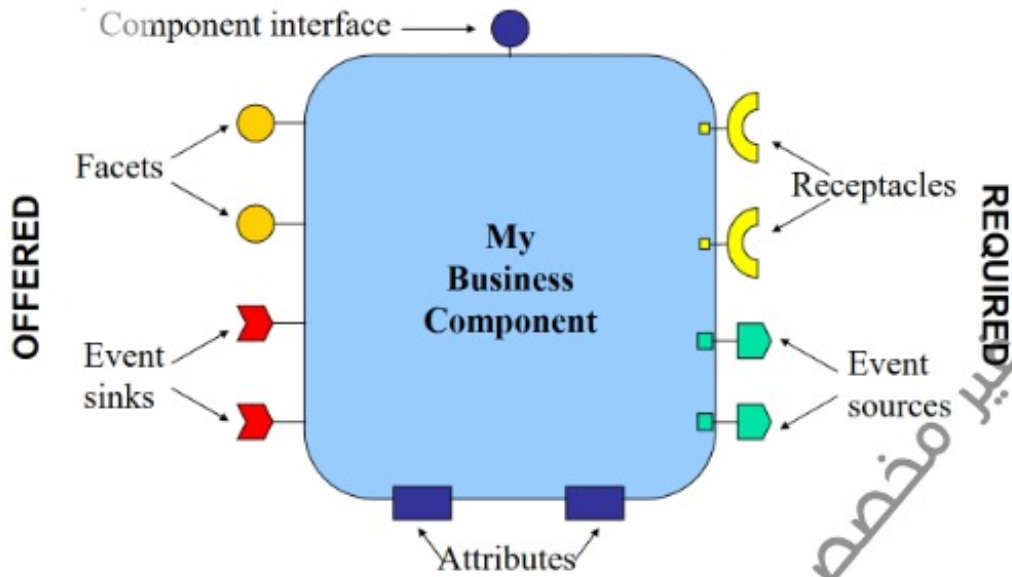
ملحوظة: يفهم المكون البرمجي عند استخدام الأنظمة الموزعة .

- ال MDE ظهرت بعد المكونات البرمجية.

مثال : نظام مكتبة

في هذه المكتبة يدخل الطالب ويأخذ كتاب (يمكنه القيام بعملية بحث مؤتمتة على الحاسب)، يجلس ويقرأه على الطاولة، إن أعجبه يستطيع استعارته، وإلا يتركه على الطاولة. سندرس حالة الاستعارة، حيث يقوم بتمرير الكتاب وبطاقته للموظفة، تأخذ الموظفة بطاقته وتقوم بعمل scan لها وتأخذ الكتاب الذي يكون فيه رقم أيضاً وتقوم بعمل scan له، وهكذا تتم عملية الاستعارة. أما في حال لم يتم تسجيل هذا الكتاب عند الموظفة، فعند خروج الطالب يتحسس جهاز على الباب ذلك ويطلق صوت إنذار لأن الكتاب يحوي sim بداخله، وعند الباب يوجد حقل مغناطيسي يتركز على ال sim فتصدر الرقم المربوط مع الكتاب، فيقوم بالبحث ضمن ال DB هل هذا الكتاب معار؟ إن لم يكن معاراً فهذا الكتاب يتم أخذه بطريقة غير شرعية ويتعرض للسرقة.

قبل شرح المثال سنقوم بذكر أنواع المكونات البرمجية: CORBA Component



الرسم السابقة توضح بيئة المكون (Component) و ال Ports الخاصة به ، حيث يوجد نوعين لل Ports:

1. Required Port:

هو منفذ يمثل الخدمات أو البيانات التي يحتاجها المكون (Component) من مصدر خارجي ليعمل بشكل صحيح و يتكون من نوعين من المنافذ:

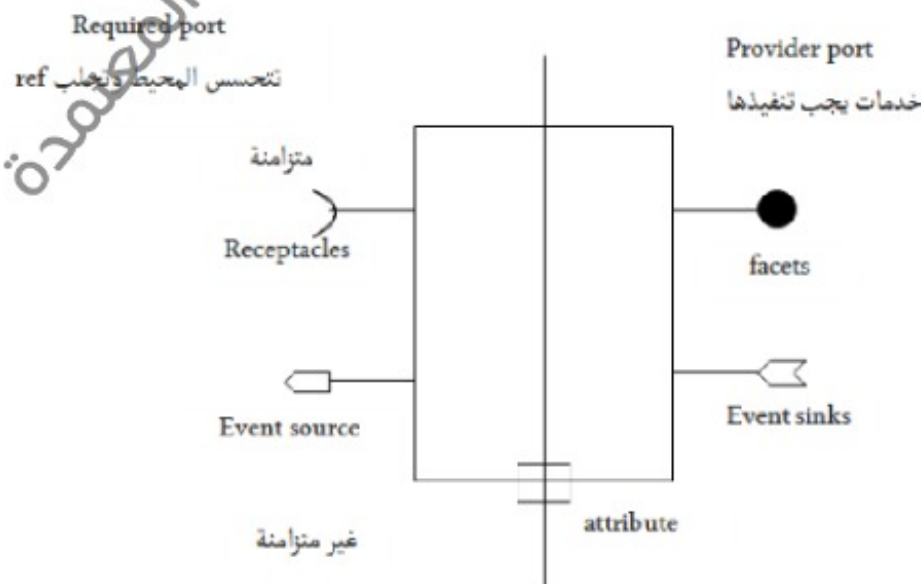
Receptacles: يسمح هذا النوع من المنافذ باستقبال خدمة من مكون آخر.
Event Source: يُستخدم لإرسال إشعارات أو أحداث إلى مكونات أخرى.

2. Provided Port:

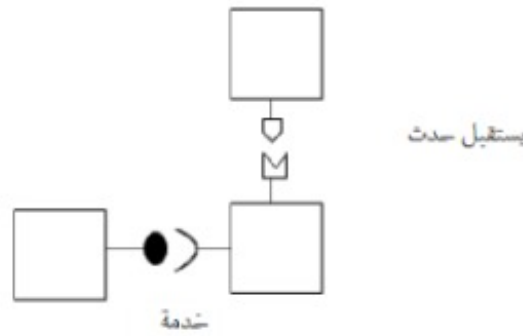
هو منفذ يمثل الخدمات أو البيانات التي يوفرها المكون لمكونات أخرى يمكنها الاستفادة منها و يتكون من نوعين من المنافذ:

Facets: هو واجهة يسمح للمكونات الأخرى بالاتصال به واستخدام وظائفه.
Event Sinks: نقطة استقبال للأحداث من مكونات أخرى.

وللفهم أكثر يمكن الاطلاع على الرسم التالية:



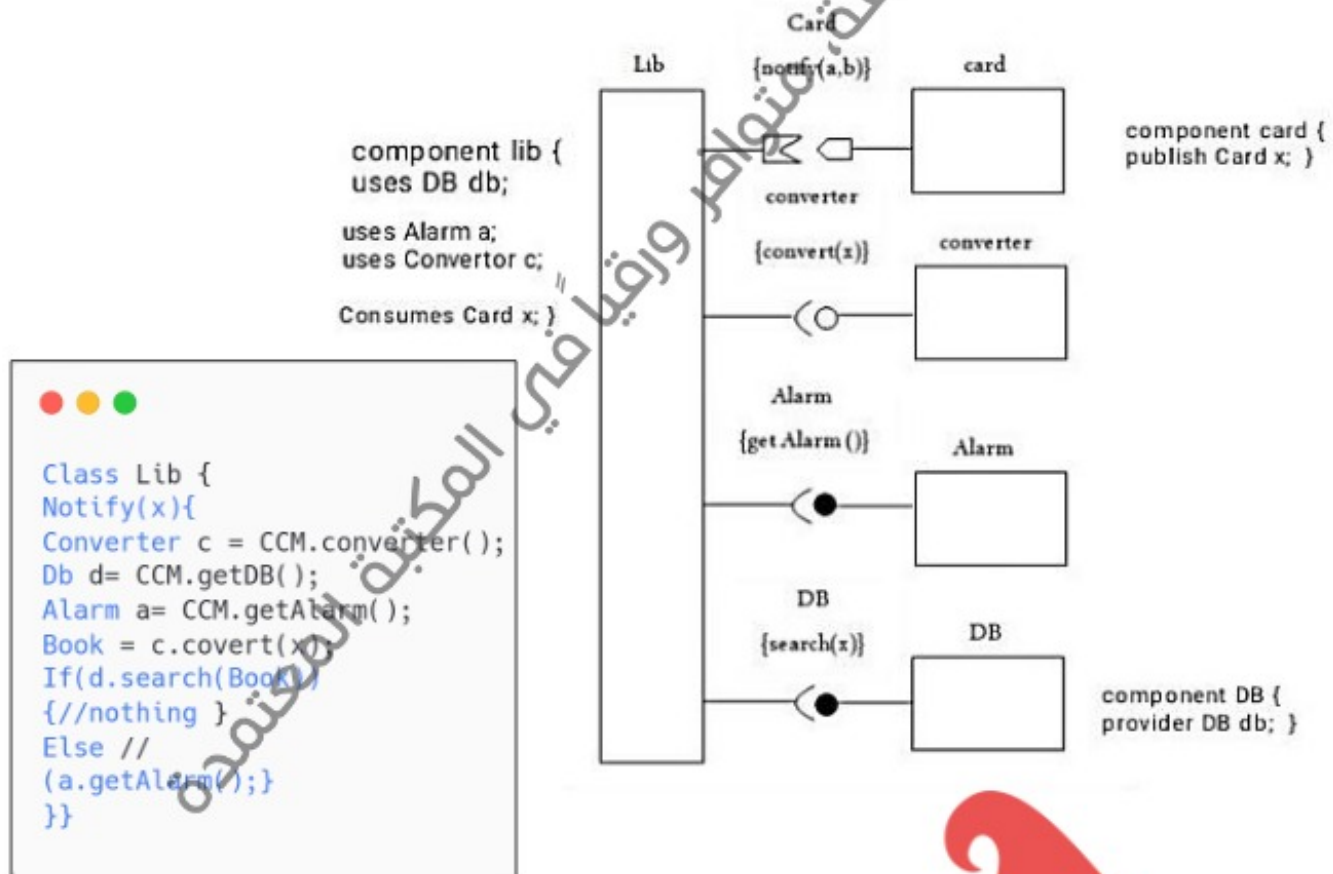
المنافذ الغير متزامنة مثل نظام الأحداث: يعمل عندما يتنفيذ حدث معين، هام جداً لكيلا يبقى النظام يعمل بشكل دائم.



بالعودة للمثال:

- على اعتبار أن ال card يجب أن تصدر رقم عند مرورها بالحقل المغناطيسي، فنختار ال port غير متزامن.
- Converter هو الذي يبدل رقم البطاقة برقم الكتاب.
- Alarm الإنذار : getAlarm.

ملاحظة هامة: المكون الجاهز لا يمكنك التعديل عليه أبداً.



وكل ما في الشكوى داء! فلن ينقذك منها إلا السير، وإتمام المهمة، والانغماس - ما استطعت - في دائرة "ما يجب، وما يجب".



■ كم Adapter تحتاج المكونات البرمجية لعمل النظام السابق؟؟

نضيف مكون برمجي واحد خاص بالمكتبة ومتصل مع أكثر من مكون، نأخذ ال reference الخاص بكل مكون ونستخدم خدماتهم.

نلاحظ أن Lib موجود لتنفيذ خدمة واحدة وهي التنبيه، وال port هو عملية أخذ reference وبالتالي يحل بمكون واحد، يستقبل حدث وصول رقم له، يتم تحويله لرقم كتاب ثم البحث عنه ضمن DB، وقد استخدم خدمات جميع المكونات الأخرى.

سؤال: لماذا غير متزامن؟

لأننا نريد أن نستدعي هذه العملية عندما يمر الكتاب من الحقل المغناطيسي.

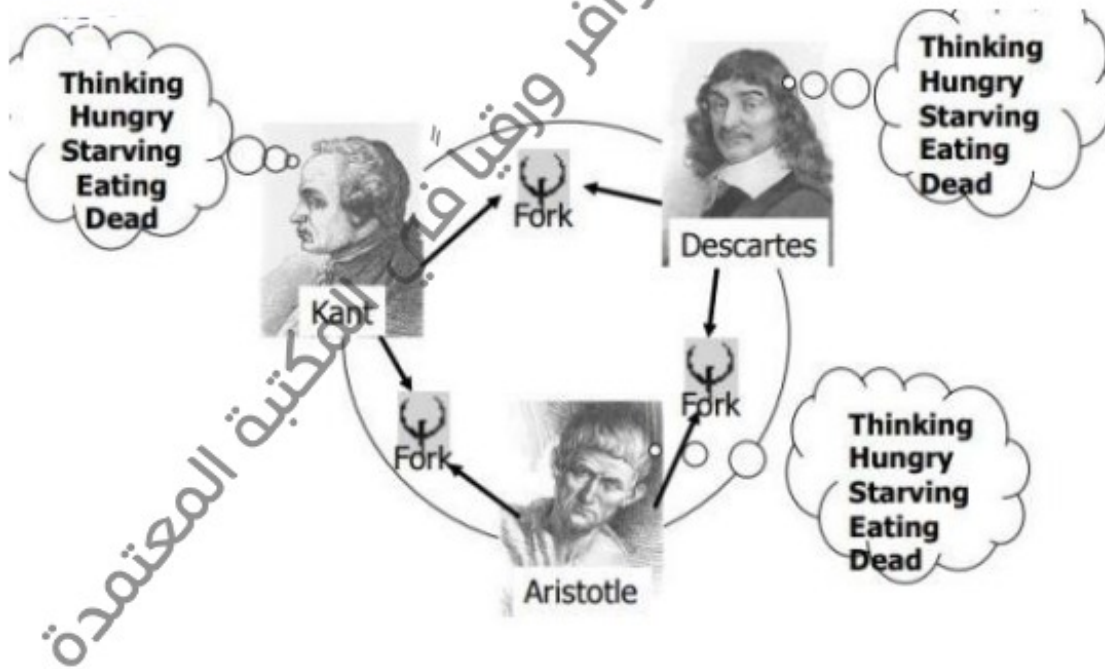
أما في حال كان متزامناً فإنه سيتم تنفيذه في كل ثانية مثلاً، سواء مر كتاب أم لم يمر. بهذه الطريقة يتحمل الحاسب عبئ عمل زائد ومن الممكن أن يمر شخص بين الثانية والأخرى.

الحياة كلها نظام أحداث.

نظام الأحداث الخاص بالمكونات البرمجية هو subject-observer Design Pattern

مثال: عشاء الفلاسفة.

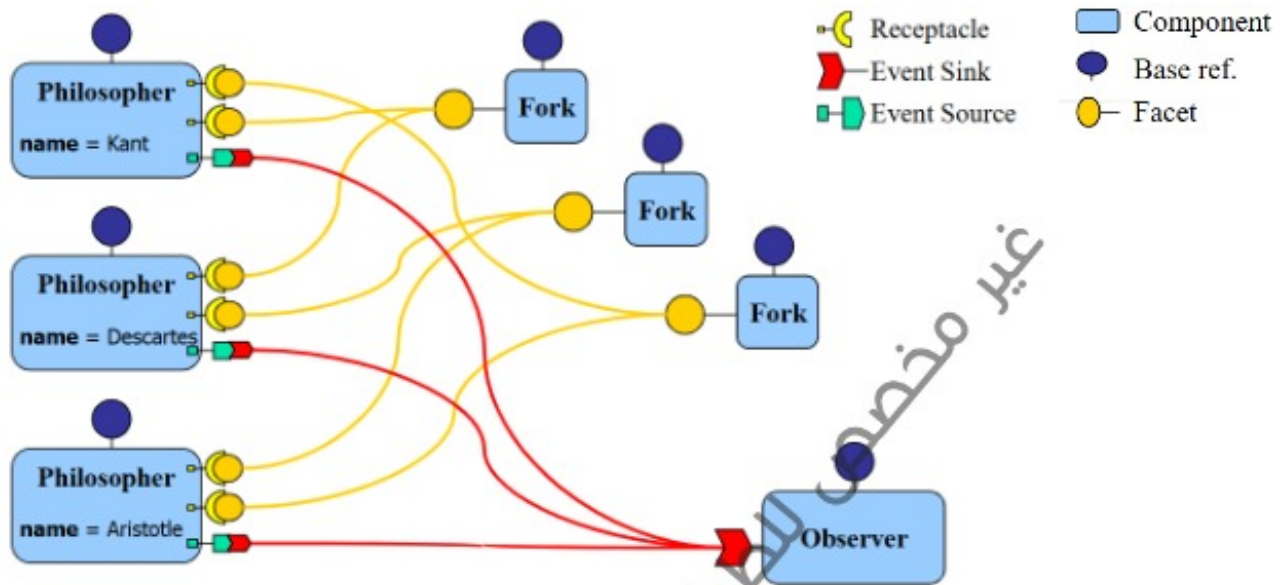
يوجد 3 فلاسفة على طاولة دائرية بحيث يوجد شوكة على يمين الفيلسوف و أخرى على يساره كما هو موضح بالشكل:



يوجد حالتان للفيلسوف:

1. إما يفكر.
2. أو يأكل (يقوم بالأكل بالشوكتين معاً، التي على يمينه وعلى يساره).

لنقم برسم ال Components الخاصة بالمسألة:



قمنا بتمثيل الفلاسفة الثلاث بالإضافة للشوك كل منها ك Component:
كل مكون فيلسوف له منفذين (منفذ للشوكة التي على يمينه ومنفذ للشوكة التي على يساره).
يوجد أيضاً مكون Observer من أجل مراقبة الحالة الحالية للفلاسفة كما سنرى لاحقاً.
لنكتب الكود الخاص بالمسألة:

```
// Importation of the Components module
// when access to OMG IDL definitions contained
// into the CCM's Components module is required.
import Components;
module DiningPhilosophers
{ // Sets the prefix of all these OMG IDL definitions.
  // Prefix generated Java mapping classes.
  typeprefix DiningPhilosophers "omg.org";
  ...
};
```

قمنا بتصدير الأدوات الخاصة ببرمجية المكونات (بما فيها OMG) ، و الكود السابق غير مطلوب بالامتحان لكن من أجل فهم التطبيق عملياً.

The Fork Interface:


```

exception InUse {};
interface Fork
{
void get() raises (InUse);
void release();
};
// The fork component.
component ForkManager
{ // The fork facet used by philosophers.
provides Fork the_fork;
};
// Home for instantiating ForkManager components.
home ForkHome manages ForkManager {};

```

واجهة الشوكة يوجد فيها تابعين **release** أي تحرير الشوكة، **get** لحجز الشوكة من قبل فيلسوف.

مكون الشوكة (**ForkManager**) يمثل شوكة يمكن أن تستخدم من قبل فيلسوف واحد فقط في كل مرة.

ForkHome لتعريف أو تهيئة أكثر من مكون للشوكة.

The philosopher State Types:


```

enum PhilosopherState
{
EATING, THINKING, HUNGRY, STARVING, DEAD
};
eventtype StatusInfo
{
public string name;
public PhilosopherState state;
public unsigned long ticks_since_last_meal;
public boolean has_left_fork;
public boolean has_right_fork;
};

```

في الكود السابق قمنا بتعريف philosopherState يحوي الحالات التي يمر بها الفيلسوف و StatusInfo يحوي معلومات الفيلسوف.

The philosopher Component

```

component Philosopher
{
  attribute string name;
  // The left fork receptacle.
  uses Fork left;
  // The right fork receptacle.
  uses Fork right;
  // The status info event source.
  publishes StatusInfo info;
};
home PhilosopherHome manages Philosopher
{
  factory new(in string name);
};
  
```

مكون الفيلسوف يحوي على اسم الفيلسوف والشوكة التي على يمينه وعلى يساره اللتان يستخدمهما، بالإضافة لحالة هذا الفيلسوف ، هذه المعلومات الموجودة في مكون الفيلسوف مهمة لأنه يجب معرفة الفيلسوف الذي يستخدم الشوك بالإضافة للشوك المستخدمة و من أجل تغيير حالة هذه الشوك (مستخدمة أم على الطاولة).

تم إنشاء PhilosopherHome من أجل إنشاء أكثر من فيلسوف (عند إنشاء فيلسوف جديد يتم تمرير اسم له).

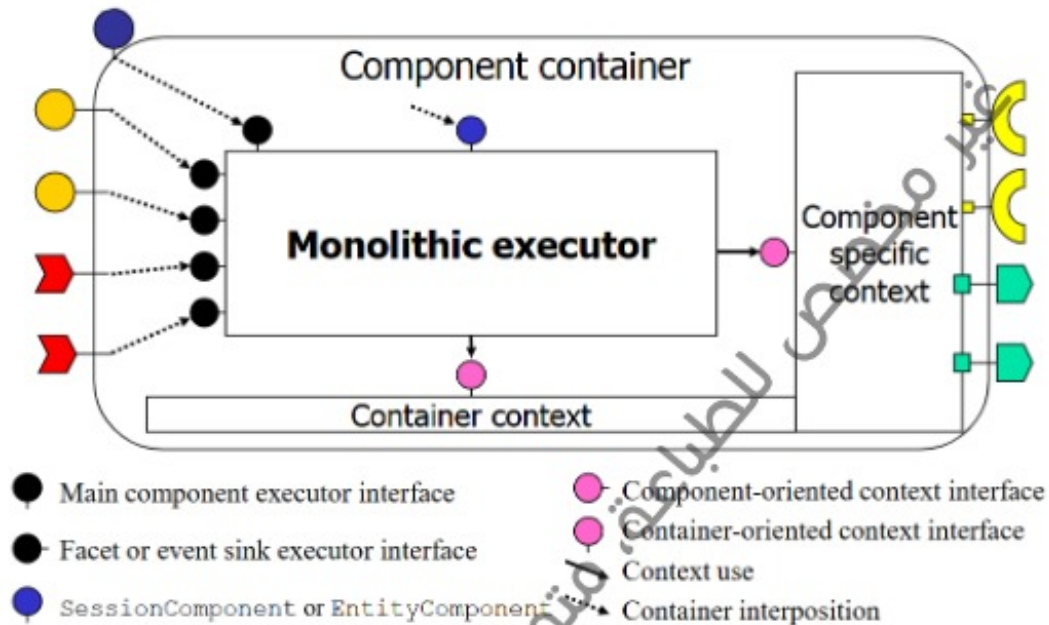
The Observer Component:

```

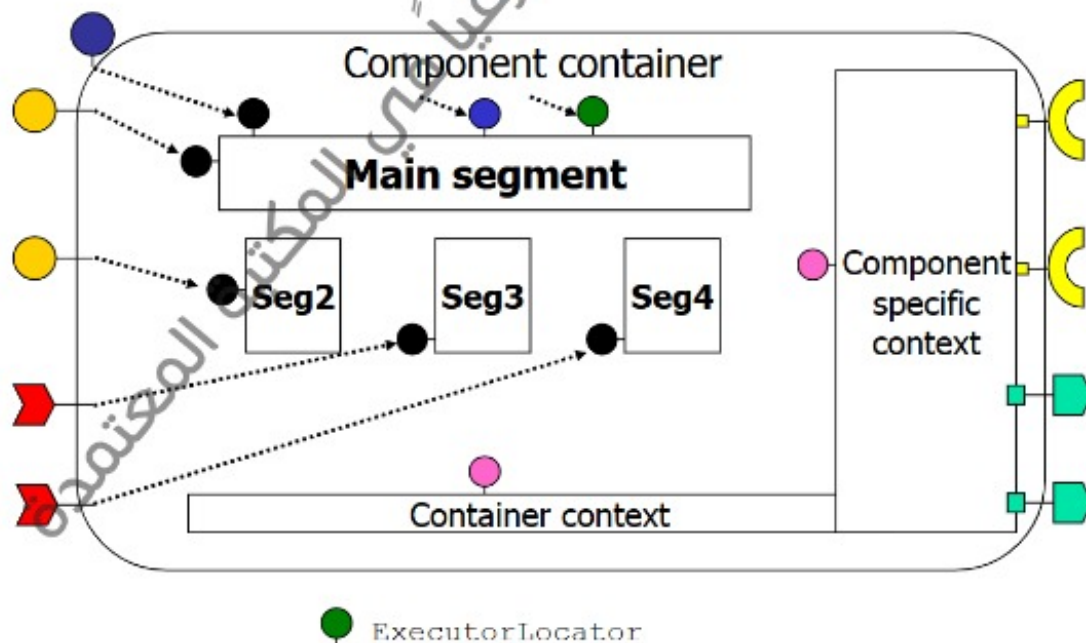
component Observer
{
  // The status info sink port.
  consumes StatusInfo info;
};
// Home for instantiating observers.
home ObserverHome manages Observer {};
  
```

في العناوين التالية سنرى رسميتين تلخصان مفهوم المكونات (Components) فهي رسومات هامة تعزز الفهم وبمجرد فهمها نكون قد فهمنا المكونات و المنافذ الخاصة بها.

A Monolithic Component Executor

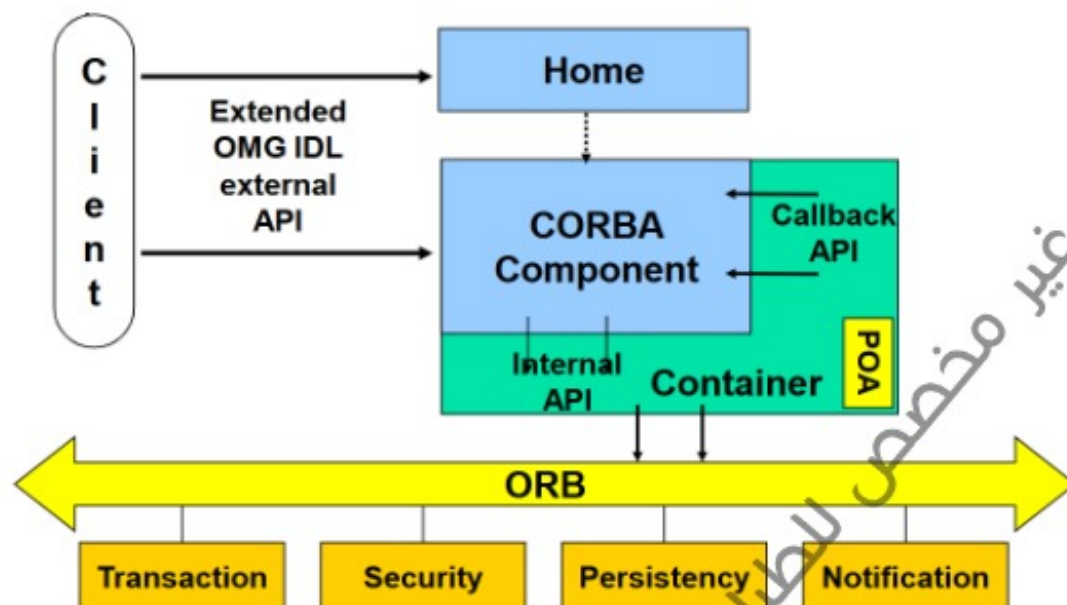


A Segmented Component Executor



تكلّمنا سابقاً أن facet و event sink لهم خصوصية لأنهم بحاجة إلى تنفيذ، وعندما قاموا بعمل الهرمية الخاصة بالمكونات البرمجية، قد صرحوا بضرورة عمل implementation لهم، وهم class ويوجد function للتنفيذ تقوم بتنفيذ ال function الذي تقدمه ال interface أو تنفيذ ال function الخاص بال notification.

The Container Architecture



لدينا عنصر مهم بال component وهو الذي يقدم التقنيات ويدير حياة الأغراض البرمجية و هو ال Container، له أنواع دخل CCM، إما أن تكون كل function داخل class واحد، أو عمل class لكل interface ويتم التعامل معه ك object.

Container View:

- A container encapsulates one or several POAs
- A container manages one kind of component
 - Entity: persistent, primary key, and explicit destruction
 - Process: persistent, no key, and explicit destruction
 - Session: exists during a session with the client
 - Service: exists during an invocation
 - EJBsession, EJBentity: for EJBs
 - Empty: user-define policy
- References are exported through FactoryFinder, Naming, or Trader services.

مزايا ال container:

1. تزويد المكون البرمجي بالمكتبات اللازمة: يتم تزويد المكون البرمجي بالمكتبات من خلال كتابة اسم المكتبة (استدعائها) داخل ملف XML ويتم تحميلها عند الإقلاع.
2. اعتراض ال method: (الفائدة إضافة خدمات)، عند تنفيذ أي غرض برمجي، الذي يقوم بتنفيذ خدمات النظام فإن ال container يقوم باعترض ال method فهذا يسمح بإضافة خدمات غير وظيفية مثل (security....)

Transaction:

وبتعريفها العام هي تنفيذ لعدد من العمليات op1,op2,op3.
يتم القيام بأي مناقلة من خلال تعريف ال operations وسلوكهم داخلها.
لكن هنا الأمر يتم بالعكس:

حيث عند تعريف ال component يوجد ملف XML وهو ملف خاص بالإعدادات configuration نقوم بإعطاء خواص لكل method، وبالتالي بدلاً من بناء transaction بالطريقة التقليدية سنقوم بذكر كل method كيف سيتم استخدامها إذا كان ضمن transaction.

خلاصة: لا أبني آليات للدارة transaction وإنما ال method بحد ذاتها كيف تتصرف عندما تكون ضمن ال transaction ولها عدة خواص:

Transaction:

- Container managed at the operation level:
 - NOT_SUPPORTED
 - REQUIRED
 - SUPPORTS
 - REQUIRES_NEW
 - MANDATORY
 - NEVER
- Self-managed using the **Components: Transaction::UserTransaction** API which Is mapped to CORBA transaction.

- Not Supported: إذا تم تحقيق ال method أولاً، لا يؤثر على ال transaction.
- Required: تنفيذ method هام في ال transaction.
- Required_New: كل الاستدعاءات داخل ال transaction تعالج بشكل تكراري إذا أخفقت.
(ننصح بمعرفة الفروق بينهم بشكل أفضل لأنه لم يتم توضيحهم أكثر ضمن المحاضرة)

ملاحظة: عند تعريف ال method لن تكون داخل transaction بشكل مباشر، إنما ستكون مستدعاة داخل operation ثانية فإن كانت required فكل method داخلها يطبق عليها سياسة ال transaction.

كيف سيكون تدخل ال container هنا؟

عند استدعاء ال operation1 الخاص بال component أذهب لملف XML الخاص بال component نفسه واقرأ خواص ال transaction ويتم تحديد سلوكه من خلال الخواص، يقوم بفتح Transaction له وكلما استدعى operation1 يقرأ خواصه.

كيف تتم عملية إدارة ال Transaction هنا؟

تتم من خلال الاستفادة من CORBA non functional service، لها مكتبة خاصة نقوم باستخدامها وهي لإدارة الخدمات الغير وظيفية.

(Real time, security, event notification, life time, ...).

بهذه الطريقة تمت إضافة المهام الغير وظيفية إلى المكونات البرمجية.

تلخيص: تم الاستفادة من اعتراض ال container لل method فيطبق ال configuration الخاص بكل method وبالمقابل أخذنا السلوك ضمن ال Transaction.

ملاحظة: هذا الأمر غير مدعوم لكل أنواع ال component فهو ضمن CORBA و factor osgi. صدر نموذج جديد لل component وهو هام للغاية.

SCA (Service Component Architecture)

وهي دمج بين ال component وال web service.

على اعتبار أن web service تعامل على أنها component لأنها خدمة متاحة للجميع.

التحزيم الخاص بالتنفيذ :packaging of implementation

بعد الانتهاء من كتابة ال component وال config الخاص به يجب أن نقوم بعملية تحزيم (تجميع) لمجموعة المكونات المرتبطة مع بعضها ويتم ذلك عن طريق أداة رسمية (tool) ويتم توليد ملف XML يوضح العلاقات:

< component C1 >

< uses username = C2 > ويتم ذكر ال interface الخاص به.

وهناك أداة تقرأ ملف التجميع Assembly وتضع كل component على أي حاسب موجود (ما هو ال IP server component).

وبالتالي بعد قراءة ملف XML فإن ال component1 موجود مثلاً على الجهاز 1، يتم التخاطب مع ال server component الخاص به ويتم عمل instance من هذا ال component مع توفير كل ما يحتاجه المكون البرمجي ليعمل (...، C++).

هذه المعلومات متوفرة بملف ال XML.

ويتم تحديد مكان المكون الثاني والثالث والرابع، وبعدها تكون عملية ال connection بينهم (تبادل مراجع) ومن ثم يتم التنفيذ.

ملاحظة: -عملية ال deployment مؤتمتة.

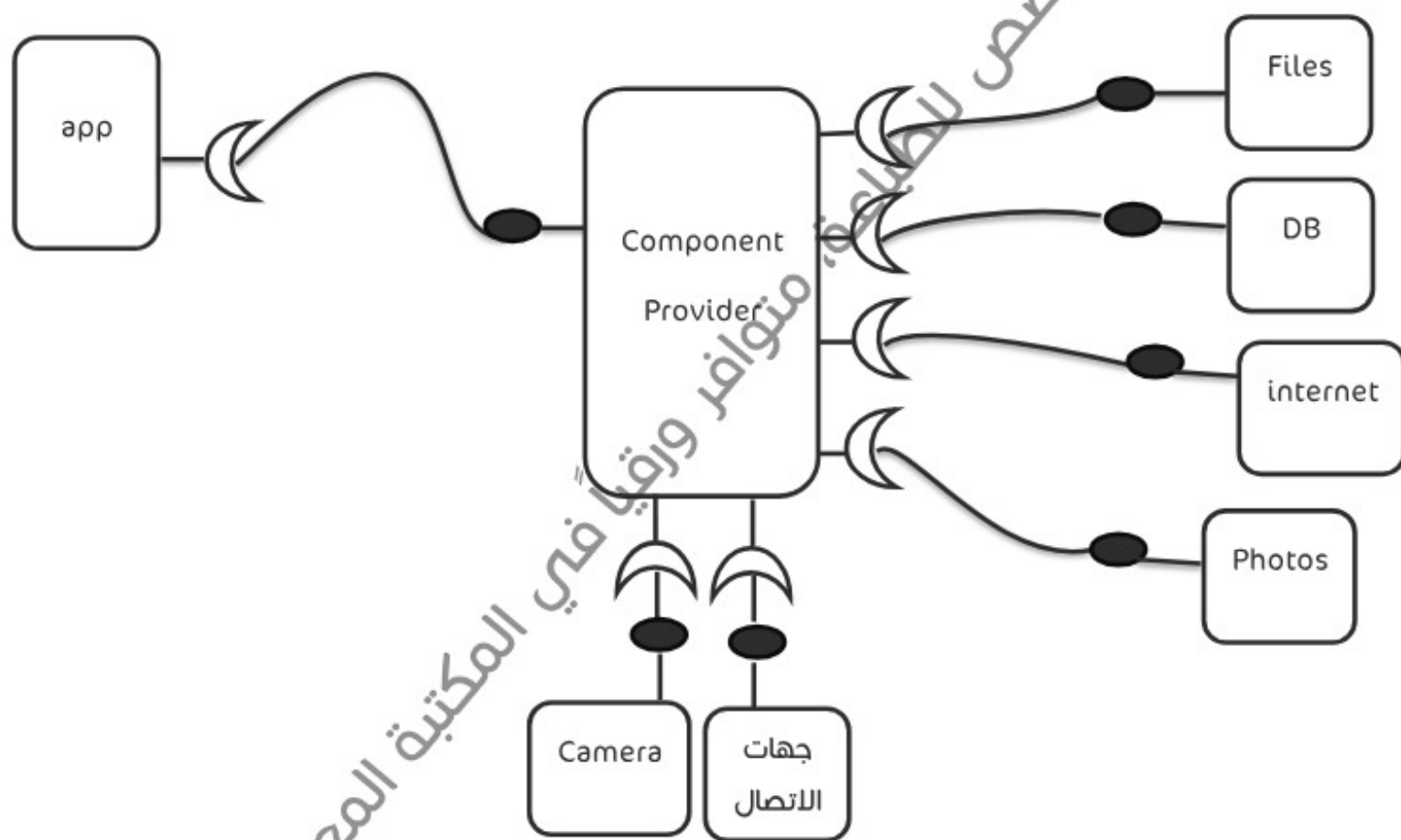
-يجب أن يوجد server component ضمن الأجهزة التي تحوي المكونات البرمجية.

حل أسئلة دورات عن ال Components

دورة 2023 الأولى:

السؤال الرابع: يبين الشكل التالي بنية تطبيقات الاندرويد، حيث يستخدم التطبيق مزود المحتوى للوصول إلى موارد الجهاز.

- 1 | ما هي الواجهة البرمجية التي تقترحها بين مكون التطبيق و مكون مزود المحتوى؟
عدل البنية السابقة كي يتم التأكد من سماحية الوصول إلى مورد معين.
- 2 | اكتب كل ما يلزم لإضافة إمكانية التأكد من سماحية الوصول إلى المورد المطلوب؟
- 3 | ماهي أفضل طريقة للتصريح عن سماحيات الوصول إلى الموارد.



الحل:

1 | الواجهة البرمجية المقترحة:

```
interface ContentProvider {
    Provider getContentProvider (ProviderTypes type);
}
```

(2) حالة الاستخدام:

يجب الحصول على الكاميرا والتقاط الصورة.

يجب الحصول على جهات الاتصال والحصول على جهة اتصال.

يجب الحصول على برنامج أو الإنترنت و إرسال الصورة عبره.

```

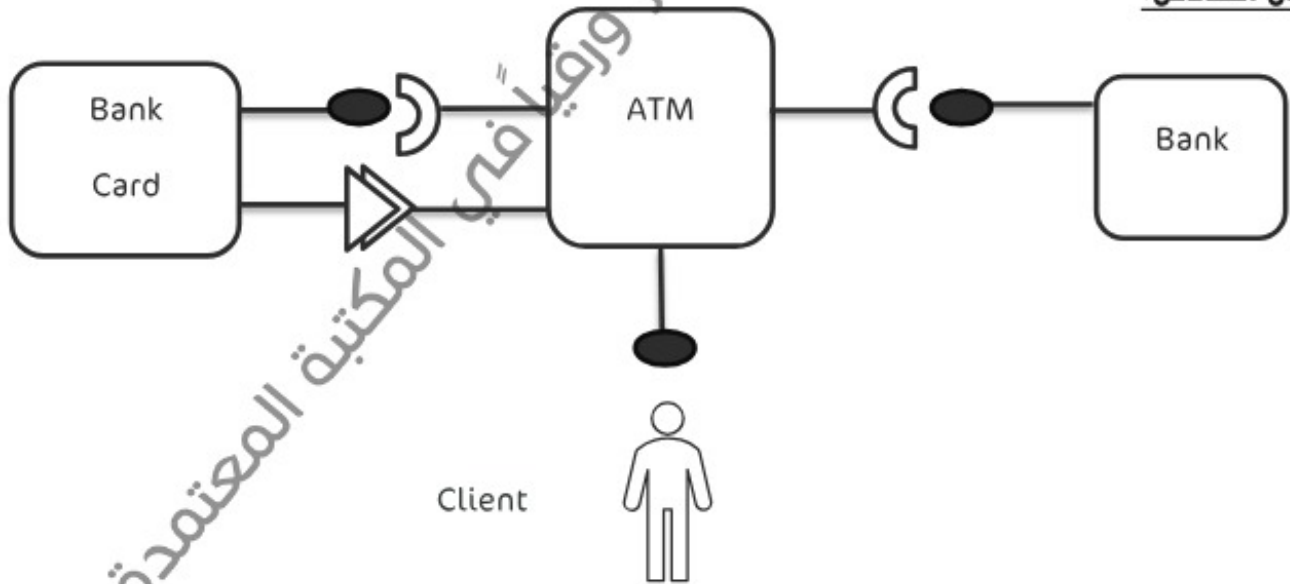
class App {
public useCase1 ( ) {
    camera = getServiceProvider(camera);
    photo = camera.getPhoto( );
    connections = getServiceProvider(connections);
    aConnection = connections.get("conn");
    programs = getServiceProvider(programs);
    whatsapp = programs.get("whatsapp");
    whatsapp.send(photo,aConnection);
}}

```

(3) أفضل طريقة للتصريح عن سمات الوصول إلى الموارد هي الوصفة type الموجودة كبارامتر في التابع .getServiceProvider

دورة 2024 الأولى:

السؤال السادس:



تبيين البنية السابقة نظام ATM: عند إدخال البطاقة، يسأل ال ATM البطاقة عن الرقم السري ورقم الحساب البنكي ويطلب من الزبون إدخال الرقم السري، عند نجاح المطابقة بين الرقم السري للبطاقة والرقم المدخل من الزبون، يمكن للزبون طلب الرصيد وسحب مبلغ من المصرف.

1. اكتب ال interfaces اللازمة لتحقيق سيناريو العمل المبين أعلاه؟

2. اكتب IDL3 لمكونات النظام؟

3. اكتب كود ATM؟

(1)

```
interface Card {
    getPIN ( );
    getAC ( );
}
```

```
interface Bank {
    getBalance(AC);
    withdraw(AC, amount);
}
```

```
interface ATM {
    getOperation ( );
    getAmount( );
}
```

(2)

```
Component ATM {
    uses Bank B;
    consume CardInfo cardInfo ;
    uses Card C;
    provides ATM atm;
}
```

```
Component BankCard {
    publishes CardInfo cardInfo ;
    provides Card C;
}
```

```
Component Bank {
    provides Bank b;
}
```

(3) كود ال ATM:

```
class ATM {
    notify(AC, PIN) {
        C = CCM.getCard( );
        B = CCM.getBank( );
        void public ATM(String PIN){ }
        PIN_N = C.getPIN();
        if(PIN_N == PIN)
            AC_N = C.getAC();
        op = getOperation( );
        if(op == "Balance")
            Balance = B.getBalance(AC_N);
        else if(op == "withdraw"){
            amount = amount.getAmount( );
            B.withdraw(AC_N, amount);
        }
    }
}
```

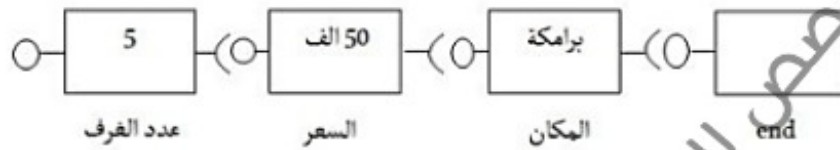
- أولاً قمنا بتعريف تابع notify لأنه يجب التأكد من وصول إشعار بوجود عميل يريد أن يقوم بالسحب أو تفقد الرصيد.
- في الباني الافتراضي يقوم العميل بإدخال الرقم السري.
- بالسطر التالي قمنا بجلب الرقم السري الصحيح الخاص بالعميل.
- يتم التحقق من أن الرقم السري الذي أدخله العميل صحيح بعدها يتم جلب العملية التي اختارها العميل ، فإن كانت تفقد الرصيد يعيد له رصيده الحالي ، أو إذا كانت العملية سحب يقوم بسحب الأموال من الرصيد الخاص به.

مثال: مكتب عقاري

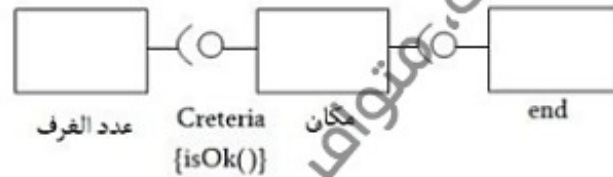
مكتب عقاري نريد البحث عن شقة بمعايير معينة.

ورد هذا المثال في الدورات ولكن كان المطلوب حله عن طريق نماذج التصميم ، لكن الدكتور قام بحله عن طريق ال Components.

أنت تريد شقة عدد غرفها أكبر من عدد معين، سعرها أقل من سعر معين وبمنطقة معينة.
أحتاج مكون (سعر، عدد غرف، مكان).
ممكن أن يأتي شخص مهتم فقط بالسعر والمكان:



أو مهتم بعدد الغرف والسعر:



نحن نريد أن يوفر المكتب لكل شخص سلسلة له.

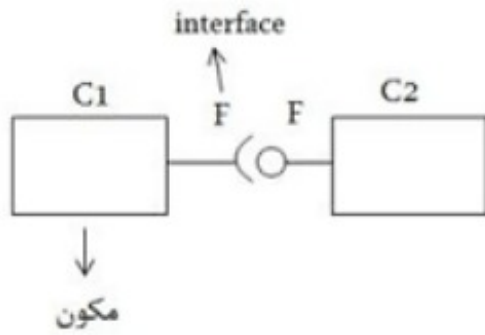
فكرة الحل أن تكون لجميع المكونات نفس ال interface ليتوافقوا مع بعض وهذا ال interface اسمه criteria وبداخله تابع اسمه isOK() تشابه ال composite أو override.



```

Class NRoom( )
{
  alt=5;
  isOk(flat){
    criteria c=CCM.getCreiteria( );
    if(alt>flat.nb)
    c.isOk(flat)
  }
}
Class end
{
  isOK(flat)
  print(flat)
}
  
```

سيبقى يسير من أجل كل عقدة، في حال تحقق الشرط ينتقل للثانية وهكذا ليصل للنهاية، وإذا لم يتحقق أحد الشروط لن يصل للنهاية وتقطع السلسلة.



يشبه ال composite بالدارات المنطقية.

انتبه من Decorator Design Pattern، ركز على أسلوب عمله.

معنى ال port: كل مكون برمجي هو class.

كل provided interface يصبح اسمه provide_f.

```
Class C2{
  f.provide_F( );
}
Class C1 {
  Connect (F f);
  get_Connection(F f);
  relese_connection( );
}
```

وجود provided interface: معناه أن لدينا عملية صريحة اسمها provide_f تعيد مرجع عن f عند عمل connection بين 2 component.

C1.connect(C2.provide_f) هنا لا نقوم بالترجمة وإنما نطلب من C1 -وهو object- أمر connect ونمرر له C2.provide_f أو في حال نريد وصله مع C3: C1.connect(C3.provide_f) وهي عملية صريحة وليست ضمنية (تبديل ref وفتح الكود) فهنا فوراً يتم استدعاؤها ضمن عملية صريحة.

وصلنا إلى نهاية المقرر
سامحونا إن وجدتم أخطاء فجلّ من لا يخطئ

وفقكم الله وفتح عليكم
إلى لقاء آخر قريب إن شاء الله

وصيتنا لكم: تعلموا وأتقنوا واعملوا بإخلاص فسوريا تحتاجنا الآن
أكثر من أي وقت مضى

بارك الله السعي وبارك الله الخُطا
والسلام 🇸🇪 🇸🇪 🇸🇪

